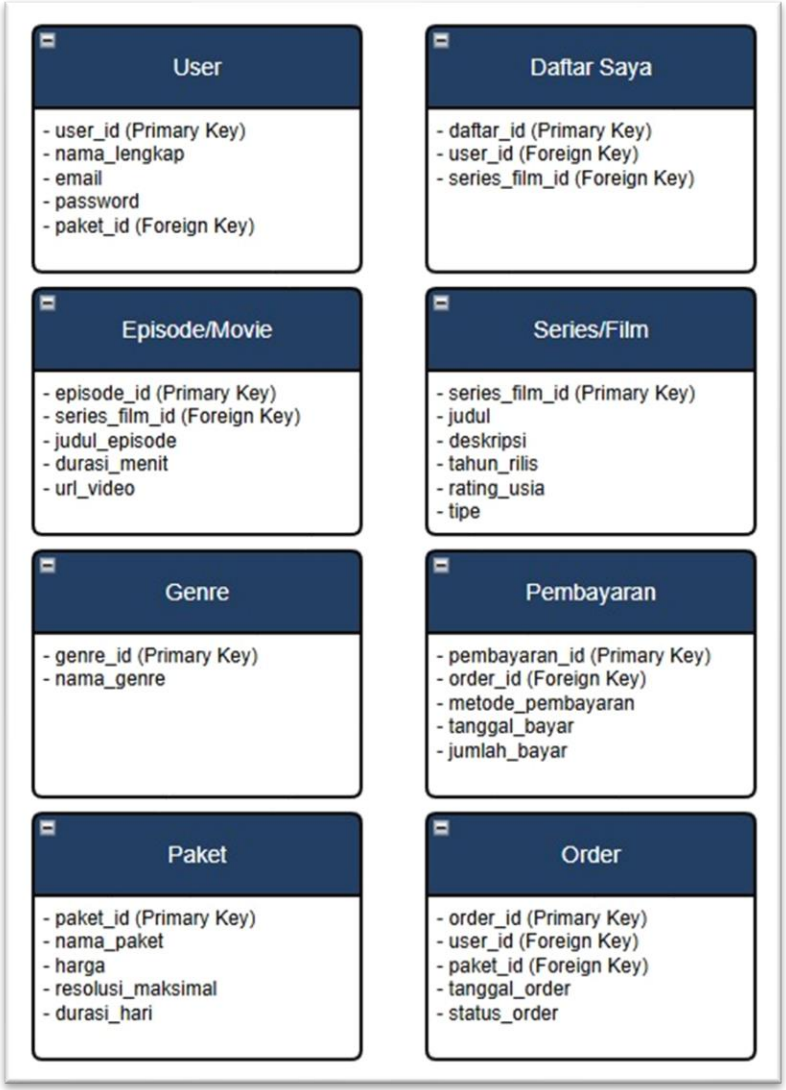
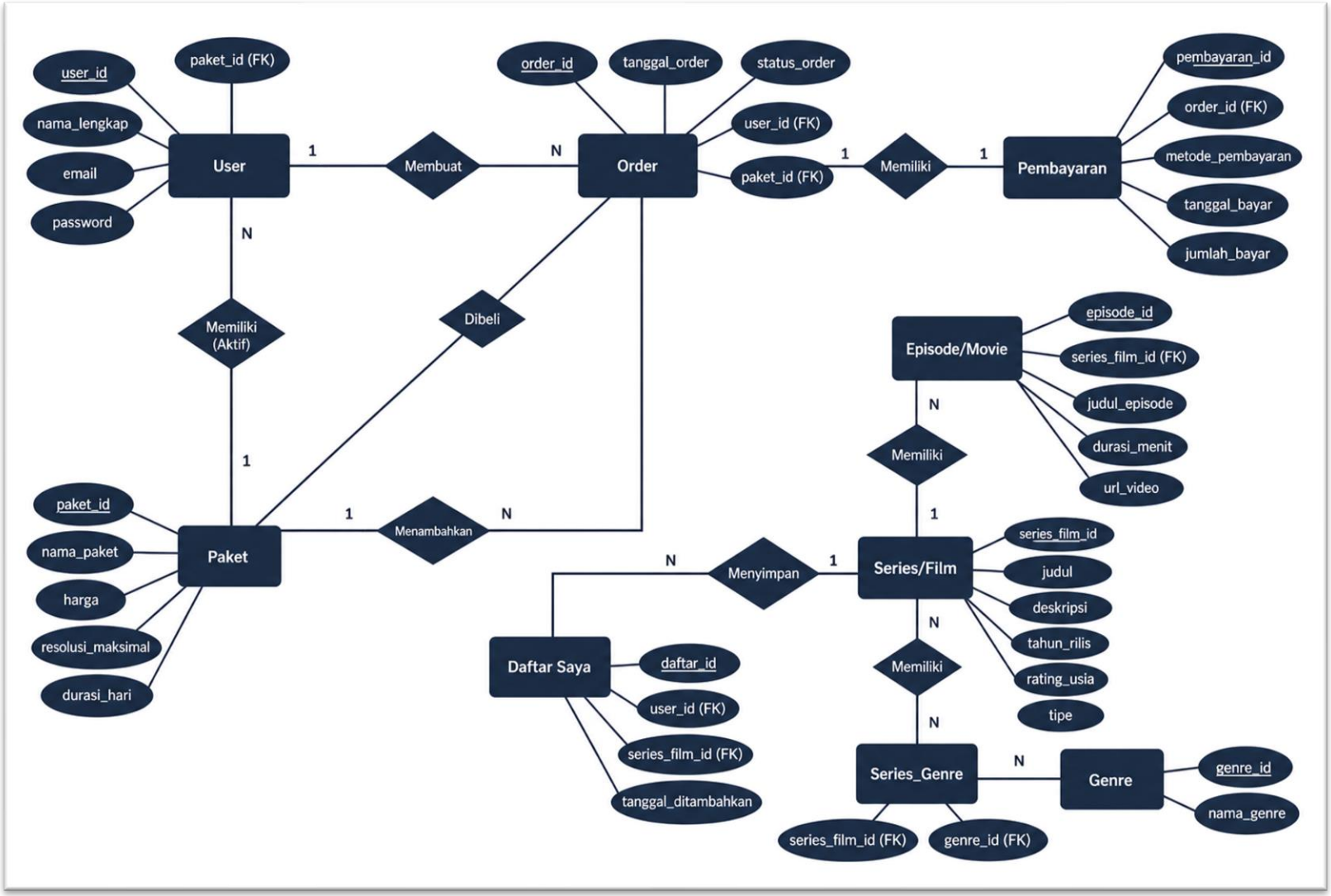
Step 1.2 : Tentukan atribut – atribut apa saja yang akan dimiliki setiap Entitas.



Step 1.3 : Menentukan Hubungan/Relasi antar entitas.

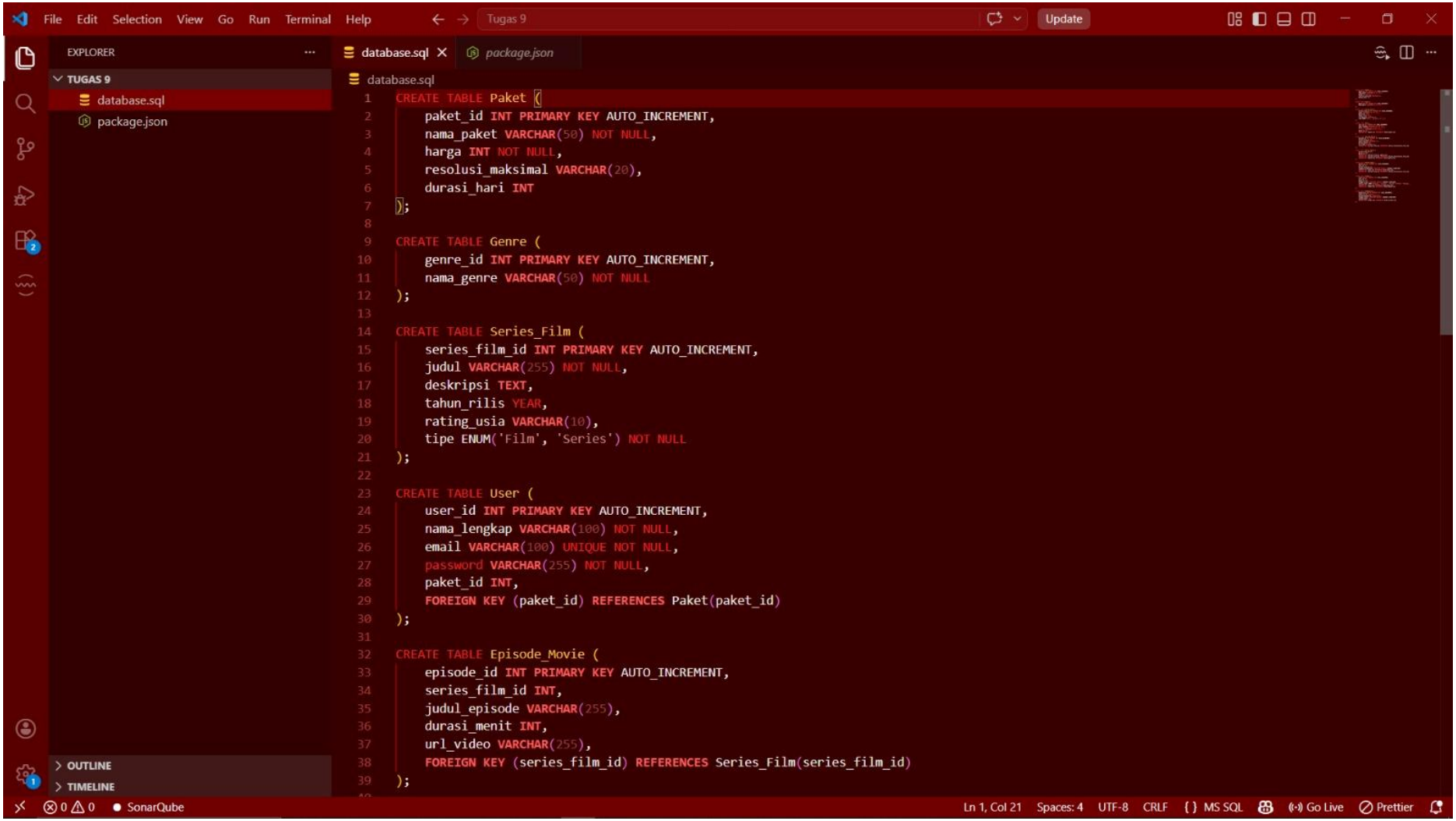
- **User ke Order (1 to Many):** Satu User bisa melakukan banyak Order (misal perpanjang langganan tiap bulan), tapi satu Order hanya dimiliki oleh satu User.
- **Paket ke Order (1 to Many):** Satu jenis Paket bisa dibeli dalam banyak Order oleh berbagai user, tapi satu Order hanya berisi satu Paket.
- **Order ke Pembayaran (1 to 1):** Satu Order memiliki tepat satu catatan Pembayaran.
- **User ke Paket (Many to 1):** Banyak User bisa berlangganan Paket yang sama (melalui relasi langsung di tabel user untuk paket yang sedang aktif).
- **Series/Film ke Episode/Movie (1 to Many):** Satu Series/Film bisa memiliki banyak Episode/Movie (jika berupa series). Jika berupa film, maka relasinya hanya berisi 1 data di tabel episode.
- **User ke Daftar Saya (1 to Many):** Satu User bisa memasukkan banyak film ke dalam Daftar Saya.
- **Series/Film ke Daftar Saya (1 to Many):** Satu Series/Film bisa dimasukkan ke Daftar Saya milik banyak user yang berbeda.
- **Series/Film ke Genre (Many to Many):** Satu Series/Film bisa memiliki banyak Genre (misal: Action & Sci-Fi), dan satu Genre bisa dimiliki banyak Series/Film. *(dengan catatan: Di langkah kedua, relasi Many-to-Many ini membutuhkan tabel bantuan tambahan, misalnya tabel Series_Genre)*.

Step 1.4 : Visualisasikan Hubungan/Relasi antar data dalam bentuk diagram ERD.



Step 2 : Preparation Database

Menerjemahkan desain ERD menjadi struktur database yang sesuai.




```sql
CREATE TABLE Paket (
    paket_id INT PRIMARY KEY AUTO_INCREMENT,
    nama_paket VARCHAR(50) NOT NULL,
    harga INT NOT NULL,
    resolusi_maksimal VARCHAR(20),
    durasi_hari INT
);

CREATE TABLE Genre (
    genre_id INT PRIMARY KEY AUTO_INCREMENT,
    nama_genre VARCHAR(50) NOT NULL
);

CREATE TABLE Series_Film (
    series_film_id INT PRIMARY KEY AUTO_INCREMENT,
    judul VARCHAR(255) NOT NULL,
    deskripsi TEXT,
    tahun_rilis YEAR,
    rating_usia VARCHAR(10),
    tipe ENUM('Film', 'Series') NOT NULL
);

CREATE TABLE User (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    nama_lengkap VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    paket_id INT,
    FOREIGN KEY (paket_id) REFERENCES Paket(paket_id)
);

CREATE TABLE Episode_Movie (
    episode_id INT PRIMARY KEY AUTO_INCREMENT,
    series_film_id INT,
    judul_episode VARCHAR(255),
    durasi_menit INT,
    url_video VARCHAR(255),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id)
);
```

File Edit Selection View Go Run Terminal Help Tugas 9 Update

EXPLORER

TUGAS 9

database.sql

package.json

database.sql | package.json

```sql
CREATE TABLE Episode_Movie (
);

CREATE TABLE Series_Genre (
    series_film_id INT,
    genre_id INT,
    PRIMARY KEY (series_film_id, genre_id),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id),
    FOREIGN KEY (genre_id) REFERENCES Genre(genre_id)
);

CREATE TABLE Daftar_Saya (
    daftar_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    series_film_id INT,
    tanggal_ditambahkan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES User(user_id),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    paket_id INT,
    tanggal_order TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status_order ENUM('Pending', 'Success', 'Failed') DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES User(user_id),
    FOREIGN KEY (paket_id) REFERENCES Paket(paket_id)
);

CREATE TABLE Pembayaran (
    pembayaran_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT UNIQUE,
    metode_pembayaran VARCHAR(50),
    tanggal_bayar TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    jumlah_bayar INT,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

OUTLINE

TIMELINE

0 0 SonarQube Ln 1, Col 21 Spaces: 4 UTF-8 CRLF { } MS SQL Go Live Prettier

*Code Log

```
CREATE TABLE Paket (
   paket_id INT PRIMARY KEY AUTO_INCREMENT,
   nama_paket VARCHAR(50) NOT NULL,
   harga INT NOT NULL,
   resolusi_maksimal VARCHAR(20),
   durasi_hari INT
);

CREATE TABLE Genre (
   genre_id INT PRIMARY KEY AUTO_INCREMENT,
   nama_genre VARCHAR(50) NOT NULL
);

CREATE TABLE Series_Film (
   series_film_id INT PRIMARY KEY AUTO_INCREMENT,
   judul VARCHAR(255) NOT NULL,
   deskripsi TEXT,
   tahun_rilis YEAR,
   rating_usia VARCHAR(10),
   tipe ENUM('Film', 'Series') NOT NULL
);

CREATE TABLE User (
   user_id INT PRIMARY KEY AUTO_INCREMENT,
   nama_lengkap VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   paket_id INT,
   FOREIGN KEY (paket_id) REFERENCES Paket(paket_id)
);
```

```sql
CREATE TABLE Episode_Movie (
    episode_id INT PRIMARY KEY AUTO_INCREMENT,
    series_film_id INT,
    judul_episode VARCHAR(255),
    durasi_menit INT,
    url_video VARCHAR(255),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id)
);

CREATE TABLE Series_Genre (
    series_film_id INT,
    genre_id INT,
    PRIMARY KEY (series_film_id, genre_id),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id),
    FOREIGN KEY (genre_id) REFERENCES Genre(genre_id)
);

CREATE TABLE Daftar_Saya (
    daftar_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    series_film_id INT,
    tanggal_ditambahkan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES User(user_id),
    FOREIGN KEY (series_film_id) REFERENCES Series_Film(series_film_id)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    paket_id INT,
    tanggal_order TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status_order ENUM('Pending', 'Success', 'Failed') DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES User(user_id),
    FOREIGN KEY (paket_id) REFERENCES Paket(paket_id)
);
```

```sql
CREATE TABLE Pembayaran (
    pembayaran_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT UNIQUE,
    metode_pembayaran VARCHAR(50),
    tanggal_bayar TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    jumlah_bayar INT,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```